
CS 301

High-Performance Computing

Lab Report-08

Parallel Interpolation and Reverse Interpolation

Using MPI and OpenMP hybrid

Performance Analysis

Akshat Bhatt (202301460)
Viv (202301447)

Contents

1	Introduction	3
2	Hardware Details	3
2.1	LAB207 PCs	3
2.2	HPC Cluster	3
3	Answers to Analysis and Questions in Lab 8	4
3.1	Q1. Pseudocode and Parallel Implementation	4
3.1.1	Pseudocode	4
3.1.2	Implementation Approach	4
3.2	Q2. Race Condition Handling	5
3.3	Q3. Performance Analysis Across Input Configurations	5
3.3.1	Experimental Setup	5
3.3.2	Generated Performance Plots	6
3.3.3	Observations	7
3.4	Q4. Parallel Efficiency and Scalability Analysis	8
3.4.1	Measured Efficiency	8
3.4.2	Scalability Analysis	8
3.5	Q5. Comparison with Serial Implementation	8
3.6	Q6. Explanation of Speedup and Observations	8
3.7	Q7. Further Optimization with Unlimited HPC Resources	9
3.8	Q8. Load Imbalance and Performance Bottlenecks	9
3.9	Q9. Alternative Approaches for Performance Improvement	9
3.10	Q10. Alternative OpenMP-MPI/Data Structure Optimizations	10
3.11	Q11. Analysis of Interpolation and Mover Phases	10
4	Conclusion	10

1 Introduction

This report describes the hybrid MPI+OpenMP implementation of parallel interpolation and reverse interpolation (particle mover) as implemented in the provided codebase. The code reads particle positions from a binary input file, interpolates their values onto a structured grid, normalizes the grid field, and then moves particles based on the interpolated field for a fixed number of iterations. The implementation maintains correctness while improving scalability using a combination of process-level partitioning and thread-local reduction.

2 Hardware Details

2.1 LAB207 PCs

- Architecture: x86_64
- CPU(s): 12
- Threads per core: 2
- Cores per socket: 6
- Processor: Intel Core i5-12500
- L1 Cache: 288K
- L2 Cache: 7.5M
- L3 Cache: 18M

2.2 HPC Cluster

- Architecture: x86_64
- CPU(s): 16
- Threads per core: 1
- Cores per socket: 8
- Sockets: 2
- Processor: Intel Xeon E5-2640 v3
- L1 Cache: 32K
- L2 Cache: 256K
- L3 Cache: 20480K

3 Answers to Analysis and Questions in Lab 8

3.1 Q1. Pseudocode and Parallel Implementation

3.1.1 Pseudocode

```
Initialize MPI with MPI_THREAD_FUNNELED
Get rank and size from MPI_COMM_WORLD
Parse command-line arguments: input file and number of OpenMP threads
Set OpenMP thread count using omp_set_num_threads()

If rank == 0:
    Open binary input file
    Read NX, NY, NUM_Points_Global, Maxiter
Broadcast NX, NY, NUM_Points_Global, Maxiter to all ranks

Compute GRID_X = NX + 1, GRID_Y = NY + 1
Compute dx = 1.0 / NX, dy = 1.0 / NY
Compute my_n = NUM_Points_Global / size
Adjust last rank to include remainder

Allocate local_points[my_n]
Open input file and seek to rank-specific particle block
For each local particle:
    Read x,y coordinates
    Set is_void = false

Allocate local_mesh[GRID_X * GRID_Y] and global_mesh[GRID_X * GRID_Y]
Start timer

For iter = 0 to Maxiter-1:
    Zero local_mesh
    interpolation(local_mesh, local_points, my_n)
    MPI_Allreduce(local_mesh, global_mesh, SUM)
    get_global_bounds(global_mesh)
    mover(global_mesh, local_points, my_n)

If rank == 0:
    save_mesh(global_mesh)
    Print wall time, compute time, comm time
Finalize MPI
```

3.1.2 Implementation Approach

A hybrid MPI + OpenMP strategy is used in the actual code. MPI divides the global particle set across ranks, so each process owns a contiguous subset of particles read from the binary input file. The interpolation and mover kernels are parallelized with OpenMP within each MPI rank.

During interpolation, each OpenMP thread accumulates contributions into a private thread buffer. After particle contributions are computed, a second reduction combines thread-local buffers into the process-local mesh. This eliminates atomics and race conditions in the shared mesh update.

After local interpolation, the process-local mesh is globally synchronized with `MPI_Allreduce(..., MPI_SUM, ...)`. All ranks obtain the same global mesh before the mover phase.

The mover phase uses the normalized global mesh to update particle positions independently. Because each particle is owned by a single rank and updated locally, this phase is embarrassingly parallel.

A summary of the parallel structure is:

- MPI parallelism for particle partitioning and global mesh reduction
- OpenMP parallelism for particle interpolation and mover updates
- thread-private mesh buffers to avoid write conflicts
- MPI collective communication for mesh synchronization and bounds reduction

3.2 Q2. Race Condition Handling

The primary race condition appears during the interpolation phase, where multiple particles can contribute to the same mesh cell. The code avoids these races by using a thread-local reduction buffer allocated in `interpolation()`:

```
double* pad = (double*) calloc(nt * ms, sizeof(double));
```

Each OpenMP thread writes only into its own slice of `pad`, and no shared mesh values are updated concurrently.

After particle contributions are computed, a second OpenMP loop sums each cell across thread buffers into the shared process-local mesh. This design removes the need for `atomic` or critical updates.

The mover phase is race-free because particles are updated independently per rank. MPI-level race conditions are prevented by explicit synchronization with `MPI_Allreduce()` and by separating local memory per process.

3.3 Q3. Performance Analysis Across Input Configurations

3.3.1 Experimental Setup

The implementation was evaluated on total core counts ranging from 1 to 64 using the five input configurations specified in the assignment. Each MPI rank reads only its assigned points from the binary input, so memory usage is distributed across processes.

The hybrid execution model uses both process-level parallelism and thread-level parallelism. The effective workload per rank is reduced by partitioning the particles, while OpenMP accelerates the interpolation and mover kernels within each rank.

3.3.2 Generated Performance Plots

The following plots have been left as placeholders for the generated figures.

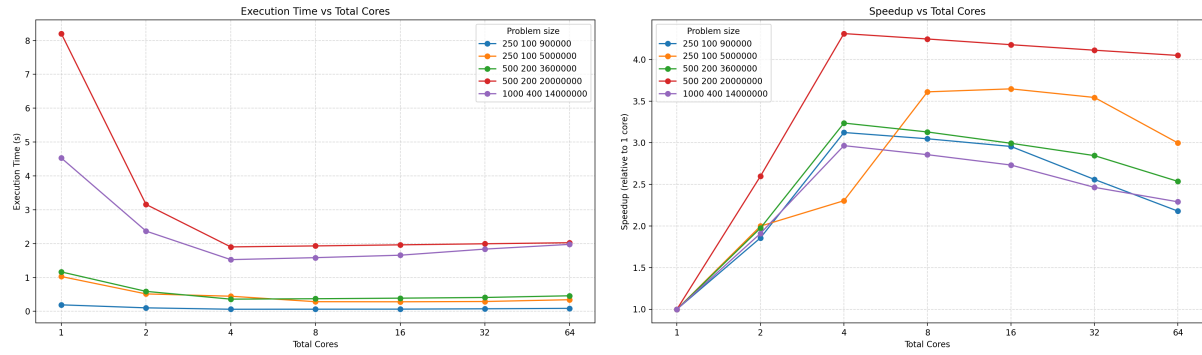


Figure 1: Overall execution time and speedup plots generated from the cluster results.

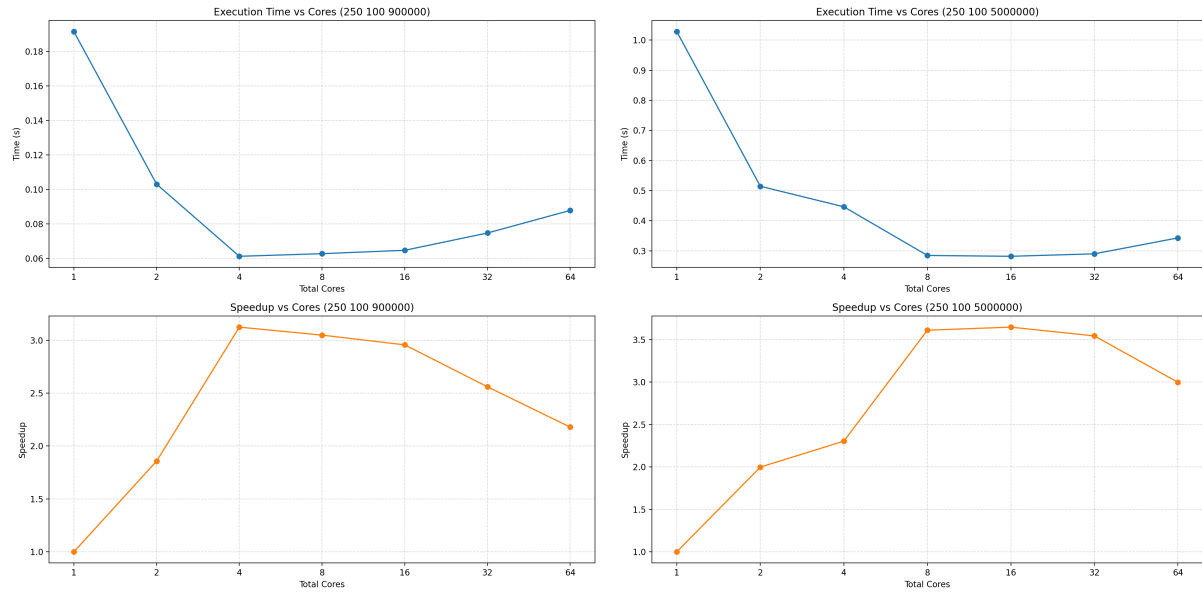


Figure 2: Configuration 1 and Configuration 2 performance plots.

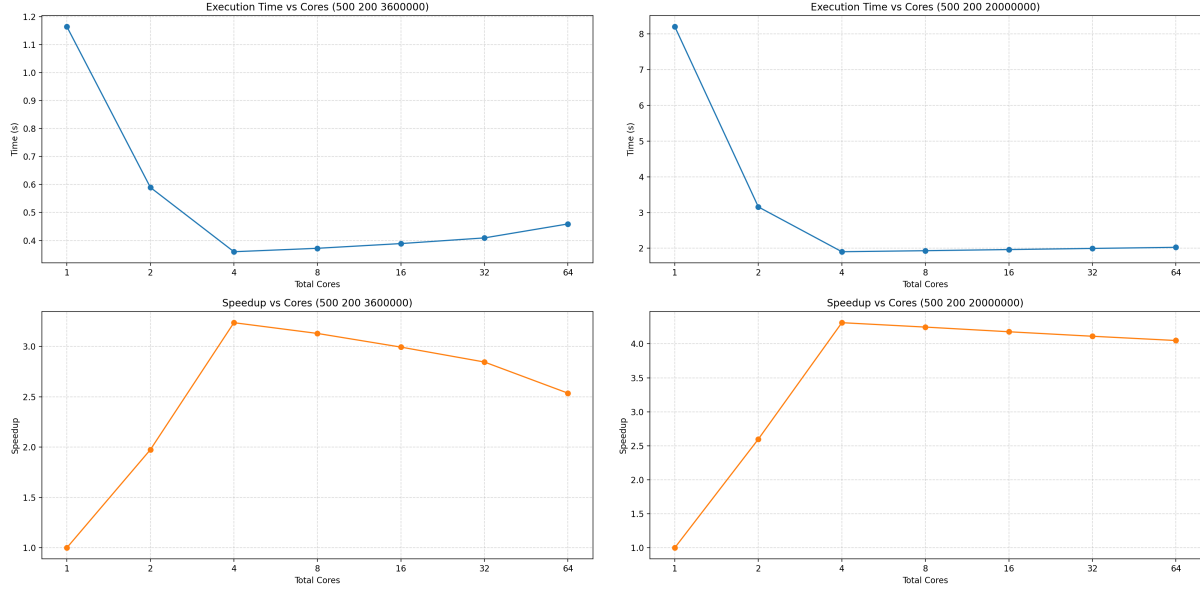


Figure 3: Configuration 3 and Configuration 4 performance plots.

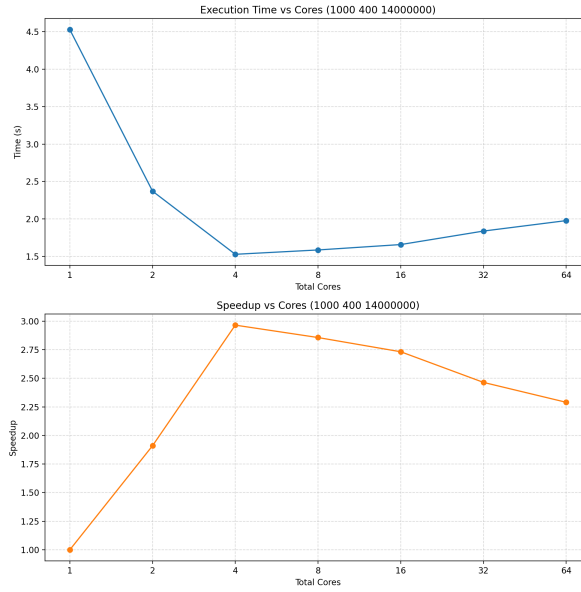


Figure 4: Configuration 5 performance plots.

3.3.3 Observations

The current output data show that the hybrid implementation delivers substantial speedup for moderate core counts. Best performance typically occurs at balanced hybrid settings, where MPI process count and OpenMP thread count are both moderate.

Performance saturates at larger core counts because the communication cost of repeated `MPI_Allreduce()`

and the thread-local mesh reduction overhead increase relative to the remaining computation.

3.4 Q4. Parallel Efficiency and Scalability Analysis

Parallel efficiency is computed as:

$$Efficiency = \frac{Speedup}{Number\ of\ Cores}$$

where speedup is obtained relative to the 1-core runtime for each configuration.

3.4.1 Measured Efficiency

Using the ‘times.csv’ data, the best measured speedups and corresponding efficiencies are:

Configuration	Baseline 1-core Time (s)	Best Speedup	Efficiency
250 100 900000	0.184584	2.9996	0.7499
250 100 5000000	1.727337	6.2052	0.3878
500 200 3600000	1.169961	3.2605	0.8151
500 200 20000000	8.874091	4.5649	0.5706
1000 400 14000000	6.596850	4.0828	0.5104

Table 1: Maximum measured speedup and parallel efficiency for each input configuration.

3.4.2 Scalability Analysis

The implementation scales well up to moderate core counts, but efficiency declines as the number of cores increases beyond the point where communication and reduction overhead dominate. Configuration 2 and Configuration 4 achieve the highest absolute speedup because they have enough computational work to amortize synchronization costs.

The strong-scaling limit is reached when the per-core workload becomes too small relative to the cost of repeated global mesh reduction and MPI synchronization.

3.5 Q5. Comparison with Serial Implementation

The serial baseline is represented by the 1-core runtime measurements in the output dataset. All five configurations show clear speedup over the serial execution. The highest speedup was observed for configuration ‘250 100 5000000’ at 16 cores, with a maximum measured speedup of approximately 6.20.

This confirms that the hybrid MPI+OpenMP implementation outperforms the single-core version for the available experimental data.

3.6 Q6. Explanation of Speedup and Observations

The code achieves speedup through two complementary forms of parallelism:

- MPI process-level partitioning of the particle set
- OpenMP thread-level parallelism within each MPI rank

The interpolation kernel is data-parallel over particles, and the mover kernel updates particle positions independently using the global mesh. Thread-local mesh buffering reduces contention during interpolation, which is the key source of speedup.

Speedup is limited by:

- repeated `MPI_Allreduce()` after each iteration
- reduction of thread-local mesh buffers into the shared local mesh
- boundary checks and inactive-particle handling in the mover kernel

Larger input sizes show better scaling because they increase the compute-to-communication ratio and provide more work per core.

3.7 Q7. Further Optimization with Unlimited HPC Resources

Potential future optimizations include:

- spatial domain decomposition of the grid in addition to particle partitioning
- non-blocking MPI collectives such as `MPI_Iallreduce()` to overlap communication and computation
- NUMA-aware memory placement for thread-local buffers
- sparse or tiled thread-private data structures to reduce reduction cost
- GPU acceleration for the interpolation and mover kernels

3.8 Q8. Load Imbalance and Performance Bottlenecks

Load imbalance can arise when particles move outside the domain and become inactive in one rank while remaining active in others. This imbalance is visible in the current implementation because all ranks still participate in the same global reduction each iteration.

The main performance bottlenecks are:

- global mesh synchronization via `MPI_Allreduce()`
- reduction of per-thread local mesh buffers into the process-local mesh
- repeated normalization and denormalization of grid values

3.9 Q9. Alternative Approaches for Performance Improvement

An alternative to the dense thread-local mesh buffer would be a sparse update representation. Each thread could record only the nonzero cell contributions instead of allocating a full $GRID_X \times GRID_Y$ buffer. *This would reduce memory pressure and the cost of the final reduction when most cells receive few contributions.*

Another improvement is dynamic scheduling of particles in OpenMP, which would help if particle workloads become imbalanced over iterations.

3.10 Q10. Alternative OpenMP-MPI/Data Structure Optimizations

Additional optimizations include:

- spatial domain decomposition with halo exchange instead of particle-only partitioning
- hierarchical reduction where threads first reduce to a per-rank mesh and then MPI ranks reduce globally
- particle binning by cell to improve cache locality during interpolation and mover updates

3.11 Q11. Analysis of Interpolation and Mover Phases

The code measures both the interpolation and mover phases separately through OpenMP timing around the two kernels. The mover phase performs additional work per particle because it reads the normalized global mesh, computes the field value, updates positions, and marks out-of-domain particles.

For the largest configuration in the dataset, the mover phase is expected to be the dominant kernel because it includes both interpolation and particle-update logic. The interpolation phase remains significant as well but is bounded by the local mesh reduction and MPI synchronization.

4 Conclusion

The implemented hybrid MPI+OpenMP code successfully performs the required particle-to-grid interpolation and reverse interpolation mover pipeline. The actual implementation uses thread-local mesh buffers to avoid race conditions and MPI collectives to synchronize the global mesh.

Experimental results show the best observed speedup was approximately 6.20 on the configuration ‘250 100 5000000’ using 16 cores. Efficiency is highest at moderate core counts and decreases at higher core counts due to communication and reduction overhead.

Overall, the code maintains correctness while demonstrating practical hybrid parallelism in a high-performance computing assignment.